F1TENTH at UCI Autonomous Racing Stack Technical Specification

Jeffery Lee
University of California, Irvine
F1TENTH at UCI

October 2026

1 Introduction

The RoboRacer (previously F1TENTH) network is a collaborative effort founded at the University of Pennsylvania and led by Professor Rahul Mangharam. The goal of RoboRacer is to provide a platform for competitions and research in the field of autonomous racing. Our team, F1TENTH at UCI, has kept the network's original name. We are part of this network and have developed our own autonomous racing stack for 1/10th scale racing. In this document, we will provide an overview and guide to our system architecture and race stack, comprising the following: the vehicle, SLAM, reinforcement learning in *WarpoRacer*, and deployment.

In a general scope, the race stack is an end-to-end autonomous navigation platform implemented in ROS 2. The sensor setup currently consists of a 2D LiDAR, plus odometry and IMU data from the VESC controller. A depth camera is mounted on the vehicle and is currently being integrated into the race stack to test for 3D perception. The vehicle carries an NVIDIA Jetson Orin Nano as its computing unit and uses the VESC controller for motor controls. Our Simultaneous Localization and Mapping (SLAM) is done by Google Cartographer (under folder `roboracer_slam`). For our autonomous racing approaches, we developed *WarpoRacer*, a highly efficient GPU-accelerated simulator written with NVIDIA Warp. *WarpoRacer* allows us to perform rapid iterations of reinforcement learning updates and PPO policy training in the setting of a competition.

2 Vehicle Hardware and Sensors

Operating 1/10th scale autonomous vehicles at speeds up to 18 m/s requires a reliable chassis. For our purpose, a well-tuned chassis is critical to preserve sensor accuracy. Minimizing the dynamic effects of aggressive turning and acceleration prevents tilting of the LiDAR and other sensors, ensuring consistent readings. This will prevent faults such as tilted LiDAR scans reading the floor as an obstacle.

Table 1: Summary of Core Compute and Perception Hardware

Category	Device	Primary Function
Compute Unit	NVIDIA Jetson Orin Nano (500 GB NVMe)	ROS 2, SLAM, and PPO policy
LiDAR	Hokuyo UST-10LX Rangefinder	2D rangefinder for object avoidance
IMU	VESC Controller IMU	Acceleration and rotation
Vision	Intel RealSense D435i	Depth camera for 3D perception

2.1 Hardware Architecture



Figure 1: The F1TENTH at UCI vehicle

- **Compute Module:** The NVIDIA Jetson Orin Nano runs Linux, and is responsible for most computation tasks that the vehicle performs.
- **Chassis:** The build is based on the 4-wheel drive Traxxas Fiesta ST Rally VXL Edition, with custom parts to allow for mounting of hardware.

- **Tires and Suspension:** The vehicle has an array of tires for varying surface conditions, and a suspension system that allows for spacing adjustments.
- **Motor Control:** We use the VESC 6 MkV TRAMPA controller to control the motor, powered by a 14.8 V (65.1 Wh) LiPo Battery. The VESC controller also has an IMU and publishes odometry data through a ROS 2 node.
- **Sensors:** The primary sensor is a Hokuyo UST-10LX LiDAR scanning a 270° by 10 m rated range (30 m maximum) at 40 Hz. The mount in the front of the vehicle also carries an Intel RealSense D435i depth camera.

3 System Architecture and Repository Structure

The complete software `race stack` repository is containerized by Docker for ease of deployment and reproducibility on various OS platforms and cloud servers. Our standard package installer is `astral-uv` for the same purpose. Setup steps are documented in the repository and can be found in the README.md file.

The software section of the document will follow the order that these scripts are run during deployment.

3.1 Simultaneous Localization and Mapping

The SLAM algorithm is currently deployed primarily for generating maps of the race track used for training the PPO policy, since the policy only requires direct observations to navigate. Localization will be integrated into the pipeline as we explore other methods in future iterations.

- `/roboracer_slam`**:** Google Cartographer is used for SLAM, and the ROS 2 node is located in this folder. The ROS 2 node subscribes to the LiDAR and IMU topics, and publishes a map of the environment. The map is generated with loop closures, and is saved to a file for later use.
- `/warpSLAM`**:** A custom SLAM implementation that utilizes NVIDIA Warp for GPU acceleration. This implementation is still under development.

3.2 Reinforcement Learning

Reinforcement learning methods, such as Proximal Policy Optimization (PPO), require up to hundreds of millions of steps to converge. Modern physics simulators such as NVIDIA Newton fall short in terms of efficiency, as they often render graphics and simulate physics not essential to autonomous racing. As a solution, we developed *WarpoRacer*, a GPU-accelerated simulator that allows for parallelized simulation of multiple vehicles and race maps. *WarpoRacer* is able to finish PPO training in under 10 minutes compared to multiple hours in standard physics simulators like PyBullet.

- `/warporacer`**:** Our core simulation engine. The PPO training loop calls the simulator for the bicycle model vehicle dynamics and collisions across 8,192 parallel instances within CUDA

kernels (tested on an NVIDIA Tesla T4 within Google Colab). NVIDIA Warp also helps accelerate the computation of LiDAR rays, which are passed into PyTorch networks for PPO training.

- **Observation Space:** The observation is a 110-element array that consists of 108 downsampled LiDAR ray casts spanning a 270° field of view along with the vehicle's current steering angle and its forward velocity. Since the PPO policy only sees readings from sensors with no map-relevant data, the method requires no localization at run time, resulting in our choice to use SLAM primarily for map generation.
- **Action Space:** There are two values that the PPO policy will output, one being the steering rate and the other being acceleration. `/warporacer_node` integrates these values into the steering angle and speed before publishing on `/drive`.
- **Reward Function:** The policy is rewarded for progressing along the track centerline with a weight dependent on forward velocity along the centerline. The policy receives penalties for driving backwards, proximity within 0.20 m of the track walls scaled by speed, and deviations off the centerline given by Frenet coordinates. Additionally, there is a set penalty for a collision, which will also end the training episode before the episode's 10,000-step limit.
- **Visualization:** The simulator is capable of outputting 2D video visualizations of the vehicle's trajectory and other statistics such as episode return. These data can be found in Weights and Biases if the option was enabled when running the training script.

3.3 Physical Vehicle Deployment

- **`/warporacer_node`:** A ROS 2 node that runs the trained PPO policy on the physical vehicle. It subscribes to the `/scan` and the `/odom` topics, and publishes drive commands to `/drive`. The observations are constructed exactly as in training, so that the policy will see matching inputs in simulation and on the physical car.
- **Launching:** Specific commands on launching the respective ROS 2 nodes for the desired task can be found in the README.md file within the repository.

3.4 Safety Constraints and Sanity Checks

Because there are many confounding factors that can cause unexpected behavior on the physical vehicle, we have implemented three safety and sanity checks.

- **`low_voltage_shutdown.py`:** A process that shuts the Jetson Orin Nano down if the LiPo battery voltage drops below 12 V.
- **`calibrate.py`:** Sends the vehicle forward for 1 m according to the VESC controller's odometry data. This is measured physically to check the VESC's speed and odometry tuning. Steering is not tuned through this script.
- **Deadman Switch:** A standalone script that runs separately from the vehicle. It is used to ensure that the vehicle will only run when the operator is actively holding down a button on the device running the script.

Additionally, we are able to tune some parameters such as vehicle speed and steering strength through environment variables without having to rebuild the Docker container.

3.5 Simulation Bridge

Since *WarpoRacer* is custom, we need a port to the official AutoDrive simulator. The `f1tenth_to_autodrive_bridge.py` script translates the topics published by `/warporacer_node` to the topics of AutoDrive's `/autodrive_f1tenth` node.

3.6 Classical Navigation Stacks

This repository also includes two other classical approaches to navigation, which are not our main research focus, but are still useful for testing purposes.

- **`/disparity_extender`:** A LiDAR-based reactive algorithm. Our implementation of disparity extenders differs by sliding a fixed window of LiDAR scans and steering towards the window with the largest disparity (farthest distance to the obstacle).
- **`/pure_pursuit`:** A controller that follows a closed race line on the track. This approach requires localization and is therefore implemented but awaiting localization integration.

4 Conclusion and Future Outlook

This documentation presented the mechanical and software foundations of the F1TENTH at UCI autonomous race stack. Through our combination of approaches, we are able to iterate a new version of the PPO policy in under 10 minutes on Google Colab using the Tesla T4 GPU. In the past, this has allowed us to train a brand new PPO policy in time when the first iteration failed unexpectedly. In the future, it is necessary to upgrade our simulator because competitions are evolving to include ramps and bridges, which *WarpoRacer* will not be able to sufficiently handle. To address this, we are currently developing *WarpTracer*, a 2.5D simulator that stacks layers of heightmaps to create a pseudo-3D environment. The simulator reuses a large part of *WarpoRacer*, from the bicycle vehicle dynamics model to NVIDIA Warp for GPU-accelerated LiDAR raycasting. Our current progress includes modeling a bridge crossing with a path underneath, and we have trained a PPO policy using this new map architecture. Next steps include scanning real tracks into the simulator, and incorporating ROS 2 to test on the physical vehicle.